

PRACTICAL WEEK 4

Task :1

Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

Code:-

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int i;
    pid_t pid, status;

    printf("Parent process PID: %d\n", getpid());

    // Create multiple child processes
    for (i = 1; i <= 3; i++) {
        pid = fork();

        if (pid < 0) {
            perror("fork failed");
            exit(1);
        }

        if (pid == 0) {
            printf("Child %d: PID = %d\n", i, getpid());
            sleep(i);
            printf("Child %d completed\n", i);
            exit(i);
        }
    }

    // Using wait()
    printf("\nParent using wait()...\n");
    pid = wait(&status);

    if (WIFEXITED(status)) {
        if (WIFEXITED(status)) {
            printf("wait(): Child PID %d finished with status %d\n",
                pid, WEXITSTATUS(status));
        }
    }

    // Using waitpid()
    printf("\nParent using waitpid()...\n");

    while ((pid = waitpid(-1, &status, 0)) > 0) {
        if (WIFEXITED(status)) {
            printf("waitpid(): Child PID %d finished with status %d\n",
                pid, WEXITSTATUS(status));
        }
    }

    printf("\nAll child processes completed.\n");

    return 0;
}
```

Output:-

```

anuan@Anushka:~$ cd ~
anuan@Anushka:~$ nano wait_waitpid.c
anuan@Anushka:~$ gcc wait_waitpid.c -o wait_waitpid
anuan@Anushka:~$ ./wait_waitpid
Parent process PID: 582
Child 1: PID = 583

Parent using wait()...
Child 2: PID = 584
Child 3: PID = 585
Child 1 completed
wait(): Child PID 583 finished with status 1

Parent using waitpid()...
Child 2 completed
waitpid(): Child PID 584 finished with status 2
Child 3 completed
waitpid(): Child PID 585 finished with status 3

All child processes completed.

```

Observation

The program successfully creates multiple child processes using `fork()`. Each child process receives a unique PID and executes independently. The `wait()` function allows the parent process to wait for a child process to complete, while `waitpid()` provides more control by allowing the parent to wait for a specific child process. The termination status of the child processes is collected successfully, demonstrating proper process synchronization between the parent and child processes.

Task 2

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

Code:-

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid;
    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0) {
        printf("Child process created\n");
        printf("Child PID: %d\n", getpid());
        printf("Child process exiting...\n");
        exit(0);
    }
    else {
        printf("Parent process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);

        printf("Parent sleeping for 20 seconds...\n");
        sleep(20);

        printf("Parent calling wait()...\n");
        wait(NULL);

        printf("Child process completed.\n");

        printf("Parent calling wait()...\n");
        wait(NULL);

        printf("Child process completed.\n");
    }

    return 0;
}

```

Output:

```
anuan@Anushka:~$ nano task2.c
anuan@Anushka:~$ gcc task2.c -o task2
anuan@Anushka:~$ ./task2
Parent process
Parent PID: 507
Child PID: 508
Parent sleeping for 20 seconds...
Child process created
Child PID: 508
Child process exiting...
ps -el | grep task2
Parent calling wait()...
Child process completed.
anuan@Anushka:~$ ps -el | grep task2
anuan@Anushka:~$
```

Observation

The program successfully creates a child process using `fork()`. The child process terminates while the parent process is still running, and its process state can be observed using the `ps` command. After the parent calls `wait()`, the child's termination status is collected and the child process is removed from the process table. This demonstrates the importance of using `wait()` for proper synchronization and management of parent and child processes.